

```

50     android:orientation="vertical"
51     android:showDividers="beginning"
52     android:divider="@android:drawable/divider_horizontal_dark">
53
54     <TextView
55         android:id="@+id/sermon_detail_textView"
56         android:layout_width="wrap_content"
57         android:layout_height="wrap_content"
58         android:text="Acts 19:1-10 - Is salvation a moment or a process? How do I know when I was
saved? Do I need to be re-baptized? Join Will Browning as he answers these questions and more while
we look at the meaning of baptism."
59         android:padding="10dp"
60         android:maxLines="100"
61         android:scrollbars="vertical"/>
62
63     <!--<ScrollView-->
64         <!--android:id="@+id/sermon_detail_textView"-->
65         <!--android:layout_width="wrap_content"-->
66         <!--android:layout_height="wrap_content"-->
67         <!--/>-->
68
69 </LinearLayout>
70
71 <LinearLayout
72     android:layout_width="match_parent"
73     android:layout_height="wrap_content"
74     android:gravity="center"
75     android:orientation="horizontal"
76     android:weightSum=".5" >
77
78     <ImageButton
79         android:id="@+id/sermon_detail_button"
80         android:layout_width="0dp"
81         android:layout_height="wrap_content"
82         android:layout_weight="0.17"
83         android:src="@android:drawable/ic_media_play"
84         android:padding="10dp" />
85
86 </LinearLayout>
87
88 </LinearLayout>

```

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <!--
3 /*
4  ** Copyright 2010, The Android Open Source Project
5  **
6  ** Licensed under the Apache License, Version 2.0 (the "License");
7  ** you may not use this file except in compliance with the License.
8  ** You may obtain a copy of the License at
9  **
10  ** http://www.apache.org/licenses/LICENSE-2.0
11  **
12  ** Unless required by applicable law or agreed to in writing, software
13  ** distributed under the License is distributed on an "AS IS" BASIS,
14  ** WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
15  ** See the License for the specific language governing permissions and
16  ** limitations under the License.
17  */
18 -->
19 <!-- Layout for SearchableActivity.
20 -->
21 <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
22     android:orientation="vertical"
23     android:layout_width="fill_parent"
24     android:layout_height="fill_parent">
25     <TextView
26         android:id="@+id/text"
27         android:textColor="?android:textColorPrimary"
28         android:textSize="17dp"
29         android:text="@string/sermon_search_instructions"
30         android:background="@android:drawable/title_bar"
31         android:layout_width="fill_parent"
32         android:layout_height="wrap_content" />
33     <ListView
34         android:id="@+id/list"
35         android:layout_width="fill_parent"
36         android:layout_height="0dp"
37         android:layout_weight="1" />
38 </LinearLayout>
39
40
41

```

```
1 <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
2   xmlns:tools="http://schemas.android.com/tools"
3   android:layout_width="match_parent"
4   android:layout_height="match_parent"
5   android:layout_marginLeft="16dp"
6   android:layout_marginRight="16dp"
7   android:baselineAligned="false"
8   android:divider="?android:attr/dividerVertical"
9   android:orientation="vertical"
10  android:showDividers="middle"
11  tools:context=".activities.SermonListActivity"
12  tools:layout="@layout/sermon_list_twopane">
13
14  <fragment android:id="@+id/sermon_detail_container"
15    android:layout_height="0dp"
16    android:name="com.chris.churchapp.fragments.SermonDetailFragment"
17    android:layout_weight="1"
18    android:layout_width="match_parent"
19    tools:layout="@layout/fragment_sermon_detail" />
20  <fragment
21    android:id="@+id/sermon_list"
22    android:name="com.chris.churchapp.fragments.SermonListFragment"
23    android:layout_width="wrap_content"
24    android:layout_height="0dp"
25    android:layout_weight="1"
26    tools:layout="@layout/activity_sermon_list"
27    android:layout_gravity="bottom" />
28
29 </LinearLayout>
```

```

1 <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
2   xmlns:tools="http://schemas.android.com/tools"
3   android:layout_width="match_parent"
4   android:layout_height="match_parent"
5   android:layout_marginLeft="16dp"
6   android:layout_marginRight="16dp"
7   android:baselineAligned="false"
8   android:divider="?android:attr/dividerVertical"
9   android:orientation="horizontal"
10  android:showDividers="middle"
11  tools:context=".SermonListActivity">
12
13  <!--
14   This layout is a two-pane layout for the Sermons
15   master/detail flow. See res/values-large/refs.xml and
16   res/values-sw600dp/refs.xml for an example of layout aliases
17   that replace the single-pane version of the layout with
18   this two-pane version.
19
20   For more on layout aliases, see:
21   http://developer.android.com/training/multiscreen/screensizes.html#TaskUseAliasFilters
22   -->
23
24  <fragment
25    android:id="@+id/sermon_list"
26    android:name="com.chris.churchapp.fragments.SermonListFragment"
27    android:layout_width="0dp"
28    android:layout_height="match_parent"
29    android:layout_weight="1"
30    tools:layout="@layout/sermon_list_item" />
31
32  <fragment android:id="@+id/sermon_detail_container"
33    android:layout_height="fill_parent"
34    android:name="com.chris.churchapp.fragments.SermonDetailFragment"
35    android:layout_weight="2"
36    android:layout_width="0dp" />
37
38  <!--<FrameLayout-->
39    <!--android:id="@+id/sermon_detail_container"-->
40    <!--android:layout_width="0dp"-->
41    <!--android:layout_height="match_parent"-->
42    <!--tools:layout="@layout/fragment_sermon_detail"-->
43    <!--android:layout_weight="2" />-->
44
45 </LinearLayout>
46

```

```
1 <resources>
2
3 <!--
4     Base application theme, dependent on API level. This theme is replaced
5     by AppBaseTheme from res/values-vXX/styles.xml on newer devices.
6     -->
7 <style name="AppBaseTheme" parent="android:Theme.Light">
8     <!--
9         Theme customizations available in newer API levels can go in
10        res/values-vXX/styles.xml, while customizations related to
11        backward-compatibility can go here.
12        -->
13 </style>
14
15 <!-- Application theme. -->
16 <style name="AppTheme" parent="AppBaseTheme">
17     <!-- All customizations that are NOT specific to a particular API-level can go here. -->
18 </style>
19
20 </resources>
21
```

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <!-- Copyright (C) 2011 The Android Open Source Project
3
4 Licensed under the Apache License, Version 2.0 (the "License");
5 you may not use this file except in compliance with the License.
6 You may obtain a copy of the License at
7
8 http://www.apache.org/licenses/LICENSE-2.0
9
10 Unless required by applicable law or agreed to in writing, software
11 distributed under the License is distributed on an "AS IS" BASIS,
12 WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
13 See the License for the specific language governing permissions and
14 limitations under the License.
15 -->
16
17 <!-- BEGIN_INCLUDE(all) -->
18 <resources>
19 <item name="activity_sermon_list" type="layout">@layout/sermon_list_onepane</item>
20 <bool name="has_two_panes">>false</bool>
21 </resources>
22 <!-- END_INCLUDE(all) -->
23
```

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <resources>
3
4 <string name="app_name">Journey Church SC</string>
5 <string name="events_section">EVENTS</string>
6 <string name="sermons_section">Sermons</string>
7 <string name="sermon_feed">http://journeymedia.libsyn.com/rssvideo</string>
8 <string name="news_section">News</string>
9 <string name="hello_world">Hello world!</string>
10 <string name="menu_settings">Settings</string>
11 <string name="title_activity_main">MainActivity</string>
12 <string name="title_activity_sermon_list">SermonActivity</string>
13 <string name="load_more">Load more -></string>
14 <string name="title_activity_player">PlayerActivity</string>
15 <string name="connection_error">Unable to load content. Check your network connection.</string>
16 <string name="xml_error">Error parsing XML.</string>
17 <string name="beliefs_section">BELIEFS</string>
18 <string name="beliefsString">Mission Statement: \nThe Journey Church exists to develop disciple-makers
19 <string name="twitter_section">Twitter</string>
20 <string name="menu_refresh">Refresh</string>
21 <string name="title_activity_sermon_detail">SermonDetailActivity</string>
22 <string name="action_settings">Settings</string>
23 <string name="menu_search">Search</string>
24 <string name="sermon_search_instructions">Search Sermons</string>
25 <string name="no_results">No results found. Please try again</string>
26
27 </resources>
```

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <resources>
3   <string-array name="main_list_items">
4     <item>Sermons</item>
5     <item>About Us</item>
6   </string-array>
7 </resources>
```



```
1 <resources>
2
3 <!--
4   Base application theme for API 11+. This theme completely replaces
5   AppBaseTheme from res/values/styles.xml on API 11+ devices.
6   -->
7 <style name="AppBaseTheme" parent="android:Theme.Holo.Light">
8   <!-- API 11 theme customizations can go here. -->
9 </style>
10
11 </resources>
12
```

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <!-- Copyright (C) 2011 The Android Open Source Project
3
4 Licensed under the Apache License, Version 2.0 (the "License");
5 you may not use this file except in compliance with the License.
6 You may obtain a copy of the License at
7
8 http://www.apache.org/licenses/LICENSE-2.0
9
10 Unless required by applicable law or agreed to in writing, software
11 distributed under the License is distributed on an "AS IS" BASIS,
12 WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
13 See the License for the specific language governing permissions and
14 limitations under the License.
15 -->
16
17 <!-- BEGIN_INCLUDE(all) -->
18 <resources>
19   <item name="activity_sermon_list" type="layout">@layout/sermon_list_onepane</item>
20   <bool name="has_two_panes">false</bool>
21 </resources>
22 <!-- END_INCLUDE(all) -->
23
```

```
1 <resources>
2
3 <!--
4   Base application theme for API 14+. This theme completely replaces
5   AppBaseTheme from BOTH res/values/styles.xml and
6   res/values-v11/styles.xml on API 14+ devices.
7   -->
8 <style name="AppBaseTheme" parent="android:Theme.Holo.Light.DarkActionBar">
9   <!-- API 14 theme customizations can go here. -->
10 </style>
11
12 </resources>
13
```

```
1 <?xml version="1.0" encoding="utf-8"?>
2
3 <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
4     xmlns:tools="http://schemas.android.com/tools"
5     android:orientation="horizontal"
6     android:layout_width="fill_parent"
7     android:layout_height="fill_parent"
8     android:weightSum="1">
9
10    <LinearLayout
11        android:orientation="vertical"
12        android:layout_width="fill_parent"
13        android:layout_height="fill_parent"
14        android:layout_weight=".5"
15        android:id="@+id/sermon_linear_layout">
16        <TextView
17            android:layout_width="wrap_content"
18            android:layout_height="wrap_content"
19            android:textAppearance="?android:attr/textAppearanceLarge"
20            android:text="This week at Journey"
21            android:id="@+id/main_title_textview"
22            android:layout_gravity="center"
23            android:gravity="center" android:padding="10dp"/>
24
25        <ImageView
26            android:layout_width="fill_parent"
27            android:layout_height="wrap_content"
28            android:id="@+id/imageView"
29            android:src="@drawable/journey_logo"
30            android:scaleType="fitCenter"
31            android:padding="10dp"
32            android:visibility="gone"/>
33
34        <TextView
35            android:layout_width="wrap_content"
36            android:layout_height="wrap_content"
37            android:textAppearance="?android:attr/textAppearanceMedium"
38            android:id="@+id/main_sermon_title_textView"
39            android:layout_gravity="center"
40            android:textStyle="bold"
41            android:gravity="center"
42            android:padding="10dp"
43            android:visibility="gone"/>
44
45        <ProgressBar
46            android:layout_width="wrap_content"
47            android:layout_height="0dp"
48            android:id="@+id/progressBar"
49            android:indeterminate="true"
```

```
50     android:visibility="visible"
51     android:layout_gravity="center"
52     android:layout_weight="1"/>
53 </LinearLayout>
54
55 <LinearLayout
56     android:orientation="vertical"
57     android:layout_width="wrap_content"
58     android:layout_height="wrap_content" android:layout_weight=".5">
59     <ListView
60         android:layout_width="fill_parent"
61         android:layout_height="wrap_content"
62         android:id="@+id/main_listView"
63         tools:listitem="@android:layout/simple_list_item_1"
64         android:choiceMode="singleChoice" />
65     </LinearLayout>
66 </LinearLayout>
```

```
1 <resources>
2
3 <!--
4     Customize dimensions originally defined in res/values/dimens.xml (such as
5     screen margins) for sw720dp devices (e.g. 10" tablets) in landscape here.
6 -->
7 <dimen name="activity_horizontal_margin">128dp</dimen>
8
9 </resources>
```

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <!-- Copyright (C) 2011 The Android Open Source Project
3
4 Licensed under the Apache License, Version 2.0 (the "License");
5 you may not use this file except in compliance with the License.
6 You may obtain a copy of the License at
7
8 http://www.apache.org/licenses/LICENSE-2.0
9
10 Unless required by applicable law or agreed to in writing, software
11 distributed under the License is distributed on an "AS IS" BASIS,
12 WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
13 See the License for the specific language governing permissions and
14 limitations under the License.
15 -->
16
17 <!-- BEGIN_INCLUDE(all) -->
18 <resources>
19   <item name="activity_sermon_list" type="layout">@layout/activity_sermon_twopane_land</item>
20   <bool name="has_two_panes">true</bool>
21 </resources>
22 <!-- END_INCLUDE(all) -->
23
```

```
1 <resources>
2
3 <!--
4     Customize dimensions originally defined in res/values/dimens.xml (such as
5     screen margins) for sw600dp devices (e.g. 7" tablets) here.
6 -->
7
8 </resources>
```



```
1 <?xml version="1.0" encoding="utf-8"?>
2 <!-- Copyright (C) 2011 The Android Open Source Project
3
4 Licensed under the Apache License, Version 2.0 (the "License");
5 you may not use this file except in compliance with the License.
6 You may obtain a copy of the License at
7
8 http://www.apache.org/licenses/LICENSE-2.0
9
10 Unless required by applicable law or agreed to in writing, software
11 distributed under the License is distributed on an "AS IS" BASIS,
12 WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
13 See the License for the specific language governing permissions and
14 limitations under the License.
15 -->
16
17 <!-- BEGIN_INCLUDE(all) -->
18 <resources>
19   <item name="activity_sermon_list" type="layout">@layout/activity_sermon_twopane</item>
20   <bool name="has_two_panes">true</bool>
21 </resources>
22 <!-- END_INCLUDE(all) -->
23
```

```

1 package com.chris.churchapp;
2
3 /*
4  * Copyright (C) 2012 The Android Open Source Project
5  *
6  * Licensed under the Apache License, Version 2.0 (the "License");
7  * you may not use this file except in compliance with the License.
8  * You may obtain a copy of the License at
9  *
10  * http://www.apache.org/licenses/LICENSE-2.0
11  *
12  * Unless required by applicable law or agreed to in writing, software
13  * distributed under the License is distributed on an "AS IS" BASIS,
14  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
15  * See the License for the specific language governing permissions and
16  * limitations under the License.
17  */
18
19 import android.content.pm.PackageManager;
20
21 //package android.support.v4.app;
22
23 import android.app.Activity;
24 import android.content.ComponentName;
25 import android.content.Context;
26 import android.content.Intent;
27 import android.content.pm.ActivityInfo;
28 import android.content.pm.PackageManager;
29 import android.content.pm.PackageManager.NameNotFoundException;
30 import android.support.v4.content.IntentCompat;
31 import android.util.Log;
32
33 /**
34  * NavUtils provides helper functionality for applications implementing
35  * recommended Android UI navigation patterns. For information about recommended
36  * navigation patterns see
37  * <a href="{@docRoot}guide/topics/fundamentals/tasks-and-back-stack.html">Tasks and Back Stack</a>
38  * from the developer guide and <a href="{@docRoot}design/patterns/navigation.html">Navigation</a>
39  * from the design guide.
40  */
41 public class NavUtils {
42     private static final String TAG = "NavUtils";
43     public static final String PARENT_ACTIVITY = "android.support.PARENT_ACTIVITY";
44
45     interface NavUtilsImpl {
46         Intent getParentActivityIntent(Activity activity);
47         boolean shouldUpRecreateTask(Activity activity, Intent targetIntent);
48         void navigateUpTo(Activity activity, Intent upIntent);
49         String getParentActivityName(Context context, ActivityInfo info);

```

```

50 }
51
52 static class NavUtilsImplBase implements NavUtilsImpl {
53
54     @Override
55     public Intent getParentActivityIntent(Activity activity) {
56         String parentName = NavUtils.getParentActivityName(activity);
57         if (parentName == null) return null;
58
59         // If the parent itself has no parent, generate a main activity intent.
60         final ComponentName target = new ComponentName(activity, parentName);
61         try {
62             final String grandparent = NavUtils.getParentActivityName(activity, target);
63             final Intent parentIntent = grandparent == null
64                 ? IntentCompat.makeMainActivity(target)
65                 : new Intent().setComponent(target);
66             return parentIntent;
67         } catch (NameNotFoundException e) {
68             Log.e(TAG, "getParentActivityIntent: bad parentActivityName '" + parentName +
69                 "' in manifest");
70             return null;
71         }
72     }
73
74     @Override
75     public boolean shouldUpRecreateTask(Activity activity, Intent targetIntent) {
76         String action = activity.getIntent().getAction();
77         return action != null && !action.equals(Intent.ACTION_MAIN);
78     }
79
80     @Override
81     public void navigateUpTo(Activity activity, Intent upIntent) {
82         upIntent.addFlags(Intent.FLAG_ACTIVITY_CLEAR_TOP);
83         activity.startActivity(upIntent);
84         activity.finish();
85     }
86
87     @Override
88     public String getParentActivityName(Context context, ActivityInfo info) {
89         if (info.metaData == null) return null;
90         String parentActivity = info.metaData.getString(PARENT_ACTIVITY);
91         if (parentActivity == null) return null;
92         if (parentActivity.charAt(0) == '.') {
93             parentActivity = context.getPackageName() + parentActivity;
94         }
95         return parentActivity;
96     }
97 }
98

```

```

99 // static class NavUtilsImplJB extends NavUtilsImplBase {
100 //
101 //     @Override
102 //     public Intent getParentActivityIntent(Activity activity) {
103 //         // Prefer the "real" JB definition if available,
104 //         // else fall back to the meta-data element.
105 //         Intent result = NavUtilsJB.getParentActivityIntent(activity);
106 //         if (result == null) {
107 //             result = super.getParentActivityIntent(activity);
108 //         }
109 //         return result;
110 //     }
111 //
112 //     Intent superGetParentActivityIntent(Activity activity) {
113 //         return super.getParentActivityIntent(activity);
114 //     }
115 //
116 //     @Override
117 //     public boolean shouldUpRecreateTask(Activity activity, Intent targetIntent) {
118 //         return NavUtilsJB.shouldUpRecreateTask(activity, targetIntent);
119 //     }
120 //
121 //     @Override
122 //     public void navigateUpTo(Activity activity, Intent upIntent) {
123 //         NavUtilsJB.navigateUpTo(activity, upIntent);
124 //     }
125 //
126 //     @Override
127 //     public String getParentActivityName(Context context, ActivityInfo info) {
128 //         String result = NavUtilsJB.getParentActivityName(info);
129 //         if (result == null) {
130 //             result = super.getParentActivityName(context, info);
131 //         }
132 //         return result;
133 //     }
134 // }
135
136 private static final NavUtilsImpl IMPL;
137
138 static {
139     final int version = android.os.Build.VERSION.SDK_INT;
140     // if (version >= 16) {
141     //     IMPL = new NavUtilsImplJB();
142     // } else {
143     IMPL = new NavUtilsImplBase();
144     // }
145 }
146
147 /**

```

```

148  * Returns true if sourceActivity should recreate the task when navigating 'up'
149  * by using targetIntent.
150  *
151  * <p>If this method returns false the app can trivially call
152  * {@link #navigateUpTo(Activity, Intent)} using the same parameters to correctly perform
153  * up navigation. If this method returns false, the app should synthesize a new task stack
154  * by using or another similar mechanism to perform up navigation.</p>
155  *
156  * @param sourceActivity The current activity from which the user is attempting to navigate up
157  * @param targetIntent An intent representing the target destination for up navigation
158  * @return true if navigating up should recreate a new task stack, false if the same task
159  *         should be used for the destination
160  */
161  public static boolean shouldUpRecreateTask(Activity sourceActivity, Intent targetIntent) {
162      return IMPL.shouldUpRecreateTask(sourceActivity, targetIntent);
163  }
164
165  /**
166   * Convenience method that is equivalent to calling
167   * <code>{@link #navigateUpTo(Activity, Intent)} navigateUpTo(sourceActivity,
168   * {@link #getParentActivityIntent(Activity)} getParentActivityIntent(sourceActivity)</code>.
169   * sourceActivity will be finished by this call.
170   *
171   * <p><em>Note:</em> This method should only be used when sourceActivity and the corresponding
172   * parent are within the same task. If up navigation should cross tasks in some cases, see
173   * {@link #shouldUpRecreateTask(Activity, Intent)}.</p>
174   *
175   * @param sourceActivity The current activity from which the user is attempting to navigate up
176   */
177  public static void navigateUpFromSameTask(Activity sourceActivity) {
178      Intent upIntent = getParentActivityIntent(sourceActivity);
179
180      if (upIntent == null) {
181          throw new IllegalArgumentException("Activity " +
182              sourceActivity.getClass().getSimpleName() +
183              " does not have a parent activity name specified." +
184              "(Did you forget to add the android.support.PARENT_ACTIVITY <meta-data> " +
185              "element in your manifest?");
186      }
187
188      navigateUpTo(sourceActivity, upIntent);
189  }
190
191  /**
192   * Navigate from sourceActivity to the activity specified by upIntent, finishing sourceActivity
193   * in the process. upIntent will have the flag {@link Intent#FLAG_ACTIVITY_CLEAR_TOP} set
194   * by this method, along with any others required for proper up navigation as outlined
195   * in the Android Design Guide.
196   *

```

```

197  * <p>This method should be used when performing up navigation from within the same task
198  * as the destination. If up navigation should cross tasks in some cases, see
199  * {@link #shouldUpRecreateTask(Activity, Intent)}.</p>
200  *
201  * @param sourceActivity The current activity from which the user is attempting to navigate up
202  * @param upIntent An intent representing the target destination for up navigation
203  */
204  public static void navigateUpTo(Activity sourceActivity, Intent upIntent) {
205      IMPL.navigateUpTo(sourceActivity, upIntent);
206  }
207
208  /**
209   * Obtain an {@link Intent} that will launch an explicit target activity
210   * specified by sourceActivity's {@link #PARENT_ACTIVITY} <meta-data>
211   * element in the application's manifest. If the device is running
212   * Jellybean or newer, the android.parentActivityName attribute will be preferred
213   * if it is present.
214   *
215   * @param sourceActivity Activity to fetch a parent intent for
216   * @return a new Intent targeting the defined parent activity of sourceActivity
217   */
218  public static Intent getParentActivityIntent(Activity sourceActivity) {
219      return IMPL.getParentActivityIntent(sourceActivity);
220  }
221
222  /**
223   * Obtain an {@link Intent} that will launch an explicit target activity
224   * specified by sourceActivityClass's {@link #PARENT_ACTIVITY} <meta-data>
225   * element in the application's manifest.
226   *
227   * @param context Context for looking up the activity component for sourceActivityClass
228   * @param sourceActivityClass {@link java.lang.Class} object for an Activity class
229   * @return a new Intent targeting the defined parent activity of sourceActivity
230   * @throws NameNotFoundException if the ComponentName for sourceActivityClass is invalid
231   */
232  public static Intent getParentActivityIntent(Context context, Class<?> sourceActivityClass)
233      throws NameNotFoundException {
234      String parentActivity = getParentActivityName(context,
235          new ComponentName(context, sourceActivityClass));
236      if (parentActivity == null) return null;
237
238      // If the parent itself has no parent, generate a main activity intent.
239      final ComponentName target = new ComponentName(context, parentActivity);
240      final String grandparent = getParentActivityName(context, target);
241      final Intent parentIntent = grandparent == null
242          ? IntentCompat.makeMainActivity(target)
243          : new Intent().setComponent(target);
244      return parentIntent;
245  }

```

```

246
247 /**
248  * Obtain an {@link Intent} that will launch an explicit target activity
249  * specified by sourceActivityClass's {@link #PARENT_ACTIVITY} <meta-data>
250  * element in the application's manifest.
251  *
252  * @param context Context for looking up the activity component for the source activity
253  * @param componentName ComponentName for the source Activity
254  * @return a new Intent targeting the defined parent activity of sourceActivity
255  * @throws NameNotFoundException if the ComponentName for sourceActivityClass is invalid
256  */
257 public static Intent getParentActivityIntent(Context context, ComponentName componentName)
258     throws NameNotFoundException {
259     String parentActivity = getParentActivityName(context, componentName);
260     if (parentActivity == null) return null;
261
262     // If the parent itself has no parent, generate a main activity intent.
263     final ComponentName target = new ComponentName(
264         componentName.getPackageName(), parentActivity);
265     final String grandparent = getParentActivityName(context, target);
266     final Intent parentIntent = grandparent == null
267         ? IntentCompat.makeMainActivity(target)
268         : new Intent().setComponent(target);
269     return parentIntent;
270 }
271
272 /**
273  * Return the fully qualified class name of sourceActivity's parent activity as specified by
274  * a {@link #PARENT_ACTIVITY} <meta-data> element within the activity element in
275  * the application's manifest.
276  *
277  * @param sourceActivity Activity to fetch a parent class name for
278  * @return The fully qualified class name of sourceActivity's parent activity or null if
279  *         it was not specified
280  */
281 public static String getParentActivityName(Activity sourceActivity) {
282     try {
283         return getParentActivityName(sourceActivity, sourceActivity.getComponentName());
284     } catch (NameNotFoundException e) {
285         // Component name of supplied activity does not exist...?
286         throw new IllegalArgumentException(e);
287     }
288 }
289 /**
290  * Return the fully qualified class name of a source activity's parent activity as specified by
291  * a {@link #PARENT_ACTIVITY} <meta-data> element within the activity element in
292  * the application's manifest. The source activity is provided by componentName.
293  *
294  * @param context Context for looking up the activity component for the source activity

```

```
295  * @param componentName ComponentName for the source Activity
296  * @return The fully qualified class name of sourceActivity's parent activity or null if
297  * it was not specified
298  */
299  public static String getParentActivityName(Context context, ComponentName componentName)
300      throws NameNotFoundException {
301      PackageManager pm = context.getPackageManager();
302      ActivityInfo info = pm.getActivityInfo(componentName, PackageManager.GET_META_DATA);
303      String parentActivity = IMPL.getParentActivityName(context, info);
304      return parentActivity;
305  }
306
307  /** No instances! */
308  private NavUtils() {
309  }
310 }
```



```

1  /**
2   *
3   */
4  package com.chris.churchapp;
5
6  import android.app.Application;
7  import android.content.Context;
8  import android.content.Intent;
9  import android.content.SharedPreferences;
10 import android.content.SharedPreferences.OnSharedPreferenceChangeListener;
11 import android.net.ConnectivityManager;
12 import android.net.NetworkInfo;
13 import android.preference.PreferenceManager;
14 import android.util.Log;
15 import android.widget.Toast;
16
17 import com.nostra13.universalimageloader.core.DisplayImageOptions;
18 import com.nostra13.universalimageloader.core.ImageLoader;
19 import com.nostra13.universalimageloader.core.ImageLoaderConfiguration;
20
21 public class ChurchApp extends Application implements OnSharedPreferenceChangeListener {
22     private static final String TAG = ChurchApp.class.getSimpleName();
23     private boolean serviceRunning;
24     // private SermonData sermonData;
25     private SharedPreferences prefs;
26
27     @Override
28     public void onCreate() {
29         super.onCreate();
30
31         ConnectivityManager connMgr = (ConnectivityManager)
32             getSystemService(Context.CONNECTIVITY_SERVICE);
33         NetworkInfo networkInfo = connMgr.getActiveNetworkInfo();
34         if (networkInfo != null && networkInfo.isConnected()) {
35             // fetch data
36             Intent refreshIntent = new Intent(getApplicationContext(), RefreshService.class);
37             startService(refreshIntent);
38         } else {
39             // show error
40             Toast.makeText(
41                 this,
42                 getResources().getString(R.string.connection_error),
43                 Toast.LENGTH_LONG).show();
44         }
45
46         // Create default options which will be used for every
47         // displayImage(...) call if no options will be passed to this method
48         DisplayImageOptions defaultOptions = new DisplayImageOptions.Builder()
49             .cacheInMemory()

```

```
50     .cacheOnDisc()
51     .build();
52     ImageLoaderConfiguration config = new ImageLoaderConfiguration.Builder(getApplicationContext())
53     .defaultDisplayImageOptions(defaultOptions)
54     .build();
55     ImageLoader.getInstance().init(config);
56
57     this.prefs = PreferenceManager.getDefaultSharedPreferences(this);
58     this.prefs.registerOnSharedPreferenceChangeListener(this);
59     // this.sermonData = new SermonData();
60     Log.i(TAG, "onCreated");
61 }
62
63 @Override
64 public void onSharedPreferenceChanged(SharedPreferences sharedPreferences,
65     String key) {
66     // TODO Auto-generated method stub
67 }
68 // public boolean isServiceRunning() {
69 //     return serviceRunning;
70 // }
71 // public void setServiceRunning(boolean serviceRunning) {
72 //     this.serviceRunning = serviceRunning;
73 // }
74 // public SermonData getSermonData() {
75 //     return sermonData;
76 // }
77 // public SharedPreferences getPrefs() {
78 //     return prefs;
79 // }
80 }
81
```

```

1 //package com.chris.churchapp;
2 //
3 //import android.content.ContentValues;
4 //import android.content.Context;
5 //import android.database.Cursor;
6 //import android.database.sqlite.SQLiteDatabase;
7 //import android.database.sqlite.SQLiteOpenHelper;
8 //import android.util.Log;
9 //
10 //import com.chris.churchapp.database.DbHelper;
11 //import com.chris.churchapp.database.SermonsContract;
12 //
13 //public class SermonData {
14 //
15 //    static final String TAG = "SermonData";
16 //
17 //    public static final String DB_NAME = "sermons.db";
18 //    public static final int DB_VERSION = 1;
19 //    public static final String TABLE = "sermons";
20 //
21 //    public static final String KEY_ID = "_id";
22 //    public static final String C_TITLE = "title";
23 //    public static final String C_PUBDATE = "pubDate";
24 //    public static final String C_URL = "url";
25 //    public static final String C_THUMBNAIL_URL = "thumbnailUrl";
26 //    public static final String C_DESCRIPTION = "description";
27 //    public static final String C_DURATION = "duration";
28 //    //public static final String C_SUBTITLE = "subtitle";
29 //
30 //    private static final String GET_ALL_ORDER_BY = C_PUBDATE + " DESC";
31 //
32 //    Context context;
33 //    private final DbHelper dbHelper;
34 //    SQLiteDatabase db;
35 //
36 //    public SermonData(Context context){
37 //        this.context = context;
38 //        dbHelper = new DbHelper(context);
39 //    }
40 //
41 //    public void insert (SermonItem item){
42 //        SQLiteDatabase db = dbHelper.getWritableDatabase();
43 //
44 //        ContentValues values = new ContentValues();
45 //        values.put(C_TITLE, item.title);
46 //        values.put(C_PUBDATE, item.pubDate.getTime());
47 //        values.put(C_URL, item.url);
48 //        values.put(C_THUMBNAIL_URL, item.videoThumbnailUrl);
49 //        values.put(C_DESCRIPTION, item.description);

```

```

50 // //values.put(C_DURATION, item.duration);
51 // db.insert(TABLE, null, values);
52 // }
53 //
54 // public Cursor getSermons() {
55 //     SQLiteDatabase db = this.dbHelper.getReadableDatabase();
56 //     return db.query(TABLE, null, null, null, null, null, GET_ALL_ORDER_BY);
57 //     new String[] {C_TITLE, C_PUBDATE},
58 //     null, null, null, null, null); //GET_ALL_ORDER_BY);
59 // }
60 //
61 // public Cursor query(String table, String[] columns, String selection,
62 //     String[] selectionArgs, String groupBy, String having, String orderBy) {
63 //     SQLiteDatabase db = this.dbHelper.getReadableDatabase();
64 //     return db.query(table, columns, selection, selectionArgs, groupBy, having, orderBy);
65 // }
66 //
67 // class DBHelper extends SQLiteOpenHelper {
68 //
69 //     private static final String DB_CREATE =
70 //         "create table " + SermonsContract.TABLE + " ("
71 //         + SermonsContract.Columns.ID + " integer primary key autoincrement, "
72 //         + SermonsContract.Columns.TITLE + " text not null, "
73 //         + SermonsContract.Columns.PUBDATE + " INTEGER, "
74 //         + SermonsContract.Columns.DESCRPTION + " text, "
75 //         + SermonsContract.Columns.WATCHED_STATE + " INTEGER, "
76 //         + SermonsContract.Columns.URL + " text not null, "
77 //         + SermonsContract.Columns.THUMBNAIL_URL + " text);";
78 //
79 //     public DBHelper() {
80 //         super(context, DB_NAME, null, DB_VERSION);
81 //     }
82 //
83 //     @Override
84 //     public void onCreate(SQLiteDatabase db) {
85 //         String sql = String.format("create table %s " + "(%s int primary key, %s int, %s text, %s text",
86 //             TABLE, C_TITLE, C_PUBDATE, C_SUBTITLE, C_THUMBNAIL_URL, C_TITLE, C_URL);
87 //         Log.d(TAG, "onCreate with SQL: " + DB_CREATE);
88 //         db.execSQL(DB_CREATE);
89 //     }
90 //
91 //
92 //     @Override
93 //     public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
94 //         // Usually ALTER TABLE statement
95 //         db.execSQL("drop if exists " + TABLE);
96 //         onCreate(db);
97 //     }
98 //

```

```
99  /// }  
100 //}  
101
```

```
1 package com.chris.churchapp;
2
3 import java.util.Date;
4
5 public class SermonItem {
6     public final String title;
7     public final Date pubDate;
8     public final String url;
9     public final String videoThumbnailUrl;
10    public final String description;
11
12    public SermonItem(String title, Date pubDate, String description, String url, String videoThumbnailUrl) {
13        this.title = title;
14        this.pubDate = pubDate;
15        this.description = description;
16        this.url = url;
17        this.videoThumbnailUrl = videoThumbnailUrl;
18    }
19
20    @Override
21    public String toString() {
22        return title;
23    }
24 }
25
```

```

1 package com.chris.churchapp;
2
3 import android.app.IntentService;
4 import android.app.Notification;
5 import android.content.ContentResolver;
6 import android.content.ContentValues;
7 import android.content.Intent;
8 import android.database.Cursor;
9 import android.os.IBinder;
10 import android.util.Log;
11
12 import com.chris.churchapp.database.SermonsContract;
13
14 import org.xmlpull.v1.XmlPullParserException;
15
16 import java.io.IOException;
17 import java.io.InputStream;
18 import java.net.HttpURLConnection;
19 import java.net.URL;
20 import java.util.Iterator;
21 import java.util.List;
22
23 public class RefreshService extends IntentService {
24     public static String TAG = "REFRESH_SERVICE";
25     private List<SermonItem> items;
26
27     private Notification.Builder refreshNotificationBuilder;
28     public static final int NOTIFICATION_ID = 1;
29
30     public RefreshService() {
31         super("RefreshService");
32     }
33
34     public RefreshService(String name) {
35         super(name);
36     }
37
38     @Override
39     protected void onHandleIntent(Intent intent) {
40         Log.v(TAG, "onHandleIntent");
41         try {
42             loadXmlFromNetwork(getString(R.string.sermon_feed));
43         } catch (IOException e) {
44             getResources().getString(R.string.connection_error);
45         } catch (XmlPullParserException e) {
46             getResources().getString(R.string.xml_error);
47         }
48     }
49

```

```

50  @Override
51  public IBinder onBind(Intent intent) {
52      return null;
53  }
54
55  private void addNewSermon(SermonItem item) {
56      ContentResolver cr = getContentResolver();
57
58      // Construct a where clause to make sure we don't already have this
59      // Sermon in the provider.
60      String w = SermonsContract.Columns.PUBDATE + " = " + item.pubDate.getTime();
61
62      // If the Sermon is new, insert it into the provider.
63      Cursor query = cr.query(SermonsContract.Columns.CONTENT_URI, null, w, null, null);
64
65      if (query.getCount() == 0) {
66          // sermonData.insert(item);
67
68          ContentValues values = new ContentValues();
69
70          values.put(SermonsContract.Columns.TITLE, item.title);
71          values.put(SermonsContract.Columns.PUBDATE, item.pubDate.getTime());
72          values.put(SermonsContract.Columns.DESCRPTION, item.description);
73          values.put(SermonsContract.Columns.URL, item.url);
74          values.put(SermonsContract.Columns.THUMBNAI_URL, item.videoThumbnailUrl);
75          values.put(SermonsContract.Columns.WATCHED_STATE, 0);
76
77          Log.v(TAG, "InsertURI: " + SermonsContract.Columns.CONTENT_URI.toString());
78
79          // Add the new quake to the Sermon provider.
80          cr.insert(SermonsContract.Columns.CONTENT_URI, values);
81      }
82      query.close();
83  }
84
85  public String loadXmlFromNetwork(String urlString) throws XmlPullParserException, IOException {
86      InputStream stream = null;
87      JourneyChurchXmlParser journeyChurchXmlParser = new JourneyChurchXmlParser();
88      //items = null;
89
90      try{
91          stream = downloadUrl(urlString);
92          items = journeyChurchXmlParser.parse(stream);
93
94          //test adding items to SQL
95          Iterator<SermonItem> it = items.iterator();
96          while(it.hasNext()) {
97              //sermonData.insert(it.next());
98              SermonItem temp = it.next();

```



```
99         addNewSermon(temp);
100     }
101
102     // make sure input stream is closed after app is done with it
103 } finally{
104     if(stream != null){
105         stream.close();
106     }
107 }
108 return null;
109 }
110
111 // Given a string representation of a URL, sets up a connection and gets
112 // an input stream.
113 private InputStream downloadUrl(String urlString) throws IOException {
114     URL url = new URL(urlString);
115     HttpURLConnection conn = (HttpURLConnection) url.openConnection();
116     conn.setReadTimeout(10000 /* milliseconds */);
117     conn.setConnectTimeout(15000 /* milliseconds */);
118     conn.setRequestMethod("GET");
119     conn.setDoInput(true);
120     // Starts the query
121     conn.connect();
122     InputStream stream = conn.getInputStream();
123     return stream;
124 }
125 }
126
127
```

```
1 package com.chris.churchapp;
2
3 import android.content.Context;
4 import android.net.ConnectivityManager;
5 import android.net.NetworkInfo;
6 import android.util.Log;
7
8 public class ConnectionDetector {
9
10     private Context _context;
11
12     public ConnectionDetector(Context context){
13         this._context = context;
14     }
15     public boolean isConnectingToInternet(){
16         ConnectivityManager connectivity = (ConnectivityManager) _context.getSystemService(Context.CONNECTIV
17         ITY_SERVICE);
18         if (connectivity != null)
19         {
20             NetworkInfo[] info = connectivity.getAllNetworkInfo();
21             if (info != null)
22                 for (int i = 0; i < info.length; i++)
23                     if (info[i].getState() == NetworkInfo.State.CONNECTED)
24                     {
25                         Log.d("Network", "NETWORKNAME: " + info[i].getTypeName());
26                         return true;
27                     }
28             }
29         return false;
30     }}
31
```

```

1 package com.chris.churchapp;
2
3 import android.util.Log;
4 import android.util.Xml;
5
6 import org.xmlpull.v1.XmlPullParser;
7 import org.xmlpull.v1.XmlPullParserException;
8
9 import java.io.IOException;
10 import java.io.InputStream;
11 import java.text.ParseException;
12 import java.text.SimpleDateFormat;
13 import java.util.ArrayList;
14 import java.util.Date;
15 import java.util.GregorianCalendar;
16
17 public class JourneyChurchXmlParser {
18
19     private static final String TAG = "JourneyChurchXmlParser";
20
21     // Don't use namespaces
22     private static final String ns = null;
23
24
25     public ArrayList<SermonItem> parse(InputStream in) throws XmlPullParserException, IOException {
26         try{
27             XmlPullParser parser = Xml.newPullParser();
28             parser.setFeature(XmlPullParser.FEATURE_PROCESS_NAMESPACES, false);
29             parser.setInput(in, null);
30             parser.nextTag();
31             //Log.d(TAG, "ParserFeed: " + parser.getName());
32             parser.nextTag();
33             //Log.d(TAG, "ParserFeed: " + parser.getName());
34             return readFeed(parser);
35         } finally{
36             in.close();
37         }
38     }
39
40     private ArrayList<SermonItem> readFeed(XmlPullParser parser) throws XmlPullParserException, IOException {
41         ArrayList<SermonItem> entries = new ArrayList<SermonItem>();
42
43         parser.require(XmlPullParser.START_TAG, ns, "channel");
44         while(parser.next() != XmlPullParser.END_TAG){
45             // Log.d(TAG, "ParserFeed: " + parser.getName());
46             if(parser.getEventType() != XmlPullParser.START_TAG){
47                 continue;
48             }

```

```

49     String name = parser.getName();
50     // Start by looking for the
51     if (name.equals("item")) {
52         entries.add(readSermonItem(parser));
53     } else {
54         skip(parser);
55     }
56 }
57
58 //Log.d(TAG, "END_TAG: " + XmlPullParser.END_TAG);
59 //Log.d(TAG, "END_DOCUMENT " + XmlPullParser.END_DOCUMENT);
60 return entries;
61 }
62
63 // Parses the contents of an item. If it encounters a title, summary, or link tag, hands them
64 // off to their respective &quot;read&quot; methods for processing. Otherwise, skips the tag.
65 private SermonItem readSermonItem(XmlPullParser parser) throws XmlPullParserException, IOException {
66     parser.require(XmlPullParser.START_TAG, ns, "item");
67     String title = null;
68     Date pubDate = null;
69     String summary = null;
70     String url = null;
71     String videoThumbnailUrl = null;
72     while (parser.next() != XmlPullParser.END_TAG) {
73         // Log.d(TAG, "readSermonItem: " + parser.getName());
74         if (parser.getEventType() != XmlPullParser.START_TAG) {
75             continue;
76         }
77         String name = parser.getName();
78         if (name.equals("title")) {
79             title = readTitle(parser);
80         } else if (name.equals("pubDate")) {
81             pubDate = readPubDate(parser);
82         } else if (name.equals("description")) {
83             summary = readDescription(parser);
84         } else if (name.equals("media:thumbnail")) {
85             videoThumbnailUrl = readThumbnailUrl(parser);
86         } else if (name.equals("enclosure")) {
87             url = readUrl(parser);
88         } else {
89             skip(parser);
90         }
91     }
92     return new SermonItem(title, pubDate, summary, url, videoThumbnailUrl);
93 }
94
95 // Processes title tags in the feed.
96 private String readTitle(XmlPullParser parser) throws IOException, XmlPullParserException {
97     parser.require(XmlPullParser.START_TAG, ns, "title");

```